Model Validation, Overfitting Control and Hyperparameter Tuning

A reproducible scikit-learn project on the Breast Cancer Wisconsin (Diagnostic) dataset

Project report · October 2026

Summary

This project shows how to estimate a model's real performance, detect and reduce overfitting, and tune hyperparameters without fooling yourself. A held-out test set was locked away first. Models were then compared with stratified 5-fold cross-validation, overfitting was diagnosed from the gap between training and validation scores, and hyperparameters were tuned inside pipelines. Nested cross-validation gave an honest estimate of the whole tuning procedure.

The best model was a scaled logistic regression with C = 1. It reached a cross-validated ROC AUC of 0.996 and a final test ROC AUC of 0.995. An unrestricted decision tree, by contrast, scored 1.000 on training data but only 0.918 in cross-validation, a textbook case of overfitting.

1. Objective

- Validate models in a way that gives a trustworthy estimate of future performance.
- Diagnose overfitting and underfitting, and control them.
- Tune hyperparameters systematically and report the result without optimistic bias.

2. Data and setup

The dataset has 569 samples and 30 numeric features computed from images of cell nuclei. The target is benign (63%) versus malignant (37%). It ships with scikit-learn, so no download is needed.

Item	Choice
Train / test split	455 / 114 samples, stratified, random_state = 42
Validation	Stratified 5-fold cross-validation on the training set only
Metric	ROC AUC (accuracy, precision, recall and F1 reported at the end)
Leakage control	StandardScaler placed inside a Pipeline, so it is re-fit on every training fold
Baseline	DummyClassifier (majority class), AUC 0.500

3. Method

Step 1. Split first. The test set is separated before any modelling and is used once, at the end.

Step 2. Diagnose. Each model is cross-validated with training scores returned. A large gap between training and validation score signals overfitting. Low scores on both signal underfitting.

Step 3. Control overfitting. Four tools were used: a validation curve over tree depth, a learning curve to see whether more data helps, regularization strength in logistic regression (C), and early stopping in gradient boosting.

Step 4. Tune. Grid search for logistic regression (11 values of C from 0.001 to 100) and randomized search for a random forest (15 candidates over trees, depth, leaf size and max_features).

Step 5. Nested cross-validation. The score of the winning search is optimistic because the same folds both chose and scored it. An outer 5-fold loop around an inner 3-fold search gives an unbiased estimate of the full procedure.

4. Results

4.1 Baseline and overfitting diagnosis

Model	Train AUC	CV AUC	Gap
Dummy (majority class)	0.500	0.500 ± 0.000	+0.000
Decision tree, unrestricted	1.000	0.918 ± 0.020	+0.082
Decision tree, max_depth = 3	0.986	0.931 ± 0.028	+0.054
Logistic regression, scaled	0.998	0.996 ± 0.005	+0.002

Table 1. Cross-validated ROC AUC on the training set.

The unrestricted tree memorizes the training data (AUC 1.000) and loses about eight points on unseen folds. Limiting its depth narrows the gap. The scaled logistic regression barely overfits and is the strongest model from the start.

4.2 Validation and learning curves

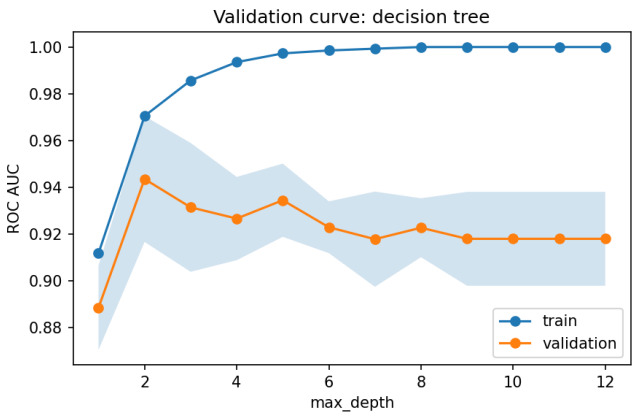


Figure 1. Decision tree: training score keeps rising with depth while validation score peaks at depth 2 and then falls.

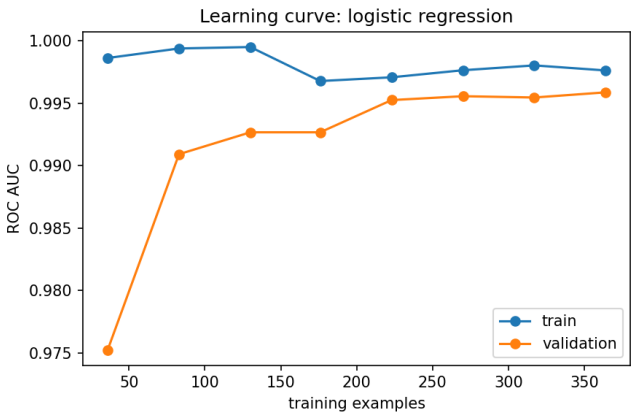


Figure 2. Logistic regression: validation score converges toward training score, so more data would add little.

In Figure 1 the training score climbs to 1.000 as depth grows, while the validation score peaks at max_depth = 2 (about 0.943) and then drifts down to about 0.918. That divergence is overfitting made

visible. In Figure 2 the two curves meet near 0.996 with a small remaining gap, so the logistic model is limited by its form, not by a lack of data. Note the narrow vertical axis: the differences are small in absolute terms.

4.3 Regularization and early stopping

C (inverse penalty)	0.001	0.01	0.1	1	10	100
CV AUC	0.986	0.993	0.995	**0.996**	0.992	0.988

Table 2. Logistic regression across regularization strengths (standard deviation 0.005 to 0.011).

Performance is best at moderate regularization (C = 1). Too strong a penalty underfits (C = 0.001) and too weak a penalty overfits (C = 100). For gradient boosting with early stopping enabled, training stopped at 76 of a possible 1000 trees once the internal validation score stopped improving.

4.4 Hyperparameter tuning and nested cross-validation

Search	Model	Best settings	CV AUC
Grid search	Logistic regression	C = 1	0.996
Randomized search (15 draws)	Random forest	n_estimators 150, max_depth 8, min_samples_leaf 3, max_features sqrt	0.990
Nested CV (outer 5, inner 3)	Logistic regression	C tuned inside each outer fold	0.996 ± 0.005

Table 3. Tuning results.

The tuned random forest (0.990) did not beat the simpler logistic regression (0.996). The nested estimate matches the plain search score, so tuning did not noticeably inflate the result on this dataset.

4.5 Final evaluation on the untouched test set

Class	Precision	Recall	F1	Support
Malignant	0.98	0.98	0.98	42
Benign	0.99	0.99	0.99	72
Accuracy			0.98	114

Table 4. Test-set metrics for the tuned logistic regression. Test ROC AUC = 0.995.

5. Discussion and conclusions

- Model complexity drives overfitting. The flexible tree overfit badly, while the regularized linear model did not.
- The gap between training and validation scores is the most useful single diagnostic.
- Scaling and any other preprocessing must live inside the cross-validation pipeline to avoid leakage.
- Nested cross-validation is the honest way to report a tuned model’s performance.
- The test result (0.995) agrees with cross-validation (0.996), which supports the validity of the process.

Limitations

- This dataset is easy, so scores sit near the ceiling and many differences fall within one standard deviation.
- The test set has only 114 samples, so a single misclassified case moves the metrics noticeably.
- One random seed and small search budgets were used. Repeated cross-validation and larger searches would strengthen the conclusions.

Next steps

- Replace the loader with your own dataset and rerun the script unchanged.
- Add class-imbalance handling and probability calibration if your target is skewed.
- Try a Bayesian tuner such as Optuna for larger search spaces.

Appendix. Full code (validation_project.py)

Requires numpy, scipy, matplotlib and scikit-learn. Run with `python validation_project.py`.

```
"""Model validation, overfitting control and hyperparameter tuning.

Run:  python validation_project.py
Needs: numpy, scipy, matplotlib, scikit-learn
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.stats import randint
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV, StratifiedKFold,
                                     cross_val_score, cross_validate, learning_curve,
                                     train_test_split, validation_curve)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

SEED = 42

# ---------------------------------------------------------------- 1. Data + split
# Lock the test set away FIRST. It is used exactly once, at the very end.
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
print(f"Train: {X_train.shape}  Test: {X_test.shape}  Positive rate: {y.mean():.2f}\n")


# ---------------------------------------------------------------- 2. Validation + diagnosis
def diagnose(name, model):
    """Cross-validate and report the train/validation gap (the overfitting signal)."""
    r = cross_validate(model, X_train, y_train, cv=cv, scoring="roc_auc",
                       return_train_score=True)
    tr, va = r["train_score"].mean(), r["test_score"].mean()
    print(f"{name:<30} train {tr:.3f} | CV {va:.3f} +/- {r['test_score'].std():.3f} | gap {tr - va:+.3f}")


print("== Step 1: baseline and overfitting diagnosis (ROC AUC, 5-fold CV) ==")
diagnose("Dummy (majority class)", DummyClassifier())
diagnose("Tree, unrestricted", DecisionTreeClassifier(random_state=SEED))
diagnose("Tree, max_depth=3", DecisionTreeClassifier(max_depth=3, random_state=SEED))
diagnose("Logistic, scaled", Pipeline([("sc", StandardScaler()),
                                       ("clf", LogisticRegression(max_iter=2000))]))
print("A big train-vs-CV gap means overfitting. A low score on both means underfitting.\n")


# ---------------------------------------------------------------- 3. Overfitting control
# 3a. Validation curve: one hyperparameter vs. train/validation score.
depths = np.arange(1, 13)
tr_s, va_s = validation_curve(DecisionTreeClassifier(random_state=SEED), X_train, y_train,
                              param_name="max_depth", param_range=depths, cv=cv,
                              scoring="roc_auc")
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(depths, tr_s.mean(1), "o-", label="train")
ax.plot(depths, va_s.mean(1), "o-", label="validation")
ax.fill_between(depths, va_s.mean(1) - va_s.std(1), va_s.mean(1) + va_s.std(1), alpha=0.2)
ax.set(xlabel="max_depth", ylabel="ROC AUC", title="Validation curve: decision tree")
ax.legend(); fig.tight_layout(); fig.savefig("validation_curve.png", dpi=150); plt.close(fig)
print(f"== Step 2a: best max_depth by validation score = {depths[va_s.mean(1).argmax()]} "
      f"(saved validation_curve.png)")

# 3b. Learning curve: does more data help, or is the model the problem?
sizes, tr_l, va_l = learning_curve(
    Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(max_iter=2000))]),
    X_train, y_train, cv=cv, scoring="roc_auc",
    train_sizes=np.linspace(0.1, 1.0, 8), random_state=SEED)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(sizes, tr_l.mean(1), "o-", label="train")
ax.plot(sizes, va_l.mean(1), "o-", label="validation")
```

```
ax.set(xlabel="training examples", ylabel="ROC AUC", title="Learning curve: logistic regression")
ax.legend(); fig.tight_layout(); fig.savefig("learning_curve.png", dpi=150); plt.close(fig)
print("== Step 2b: learning curve saved (learning_curve.png)")

# 3c. Regularization: smaller C = stronger penalty.
print("\n== Step 2c: regularization strength (logistic C) ==")
for C in [0.001, 0.01, 0.1, 1, 10, 100]:
    pipe = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(C=C, max_iter=2000))])
    s = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    print(f"  C={C:<7} CV AUC {s.mean():.3f} +/- {s.std():.3f}")

# 3d. Early stopping: stop adding trees when the internal validation score stalls.
gb = GradientBoostingClassifier(n_estimators=1000, learning_rate=0.05, max_depth=3,
                                validation_fraction=0.15, n_iter_no_change=10,
                                random_state=SEED).fit(X_train, y_train)
print(f"\n== Step 2d: early stopping used {gb.n_estimators_} of 1000 trees ==\n")


# ------------------------------------------------------------------ 4. Hyperparameter tuning
# The scaler lives INSIDE the pipeline so it is re-fit on each training fold (no leakage).
print("== Step 3: hyperparameter tuning ==")
log_search = GridSearchCV(
    Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(max_iter=2000))]),
    {"clf__C": np.logspace(-3, 2, 11)},
    cv=cv, scoring="roc_auc", n_jobs=-1).fit(X_train, y_train)
print(f"Grid search (logistic): best {log_search.best_params_}, CV AUC {log_search.best_score_:.3f}")

rf_space = {"n_estimators": randint(100, 300), "max_depth": randint(2, 12),
            "min_samples_leaf": randint(1, 10), "max_features": ["sqrt", "log2", 0.5]}
rf_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=SEED), rf_space, n_iter=15, cv=cv,
    scoring="roc_auc", n_jobs=-1, random_state=SEED).fit(X_train, y_train)
print(f"Random search (forest): best {rf_search.best_params_}, CV AUC {rf_search.best_score_:.3f}")

# ------------------------------------------------------------------ 5. Nested CV
# best_score_ above is optimistic because the same folds picked AND scored the winner.
# Nested CV: the inner loop tunes, the outer loop gives an honest estimate of the whole procedure.
print("\n== Step 4: nested cross-validation (unbiased estimate of the tuning procedure) ==")
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED + 1)
inner_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED + 2)
nested = cross_val_score(
    GridSearchCV(Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression(max_iter=2000))]),
                 {"clf__C": np.logspace(-3, 2, 11)}, cv=inner_cv, scoring="roc_auc"),
    X_train, y_train, cv=outer, scoring="roc_auc", n_jobs=-1)
print(f"Nested CV AUC (logistic): {nested.mean():.3f} +/- {nested.std():.3f}")

# ------------------------------------------------------------------ 6. Final test (once)
print("\n== Step 5: final evaluation on the untouched test set ==")
final = log_search.best_estimator_  # chosen using training data only
proba = final.predict_proba(X_test)[:, 1]
print(f"Test ROC AUC: {roc_auc_score(y_test, proba):.3f}")
print(classification_report(y_test, final.predict(X_test), target_names=["malignant", "benign"]))
```